Plan de test et matrice de traçabilité

Ndeye Khadidiatou LO

1. Objet et périmètre

Je teste le prototype CreditRisk, qui estime le risque de défaut de remboursement pour une institution de microfinance. Il a quatre composants : validation (`app/validation.py`), prétraitement (`app/preprocessing.py`), modèle de Machine Learning (`app/prediction.py`, artefacts `models/`) et API FastAPI (`app/main.py`).

Pour les données, j'utilise un jeu synthétique de 6 000 dossiers que je génère avec `scripts/train_model.py` (graine 42, 70 % pour l'entraînement, 30 % pour le test, 28,1 % de défauts). Il me sert à tester le système, mais il ne permet pas de juger la performance réelle d'un modèle de crédit.

Je n'ai pas couvert l'équité, la conformité réglementaire, la sécurité réseau (authentification, chiffrement) ni la validation métier du seuil de décision. J'en parle dans les limites et les recommandations du rapport de qualification.

2. Stratégie

Niveau	Fichier	Nombre de cas	Objectif
Unitaire	`tests/test_validation.py`	52	Règles de validation, alertes, prétraitement
Fonctionnel	`tests/test_api.py`	40	Contrat HTTP des trois endpoints
Intégration	`tests/test_integration.py`	22	Chaîne validation, prétraitement, modèle, API et pannes simulées
Robustesse	`tests/test_robustness.py`	85	Entrées manquantes, non numériques, non finies, limites, hors domaine, schéma incorrect, propriétés métier
Non-régression	`tests/test_regression.py`	12	Métriques, jeu « or », sauvegarde et rechargement, réentraînement
Performance	`scripts/benchmark.py`, `locustfile.py`	5 paliers + Locust	Latence, P95, débit, charge progressive
Démonstration CI	`tests/test_echec_volontaire.py`	1	Prouver qu'un test en échec bloque le workflow

J'ai utilisé les classes d'équivalence, l'analyse des valeurs limites, des tests de contrat, des pannes simulées avec monkeypatch, des tests métamorphiques pour le sens de variation du risque, un fuzzing sur 500 demandeurs avec une graine fixe, et une comparaison à une version de référence avec des tolérances fixées avant de mesurer.

3. Priorisation

Priorité	Critère de gravité d'une défaillance
P1	Décision de crédit erronée, fuite d'information technique, indisponibilité non signalée, régression du modèle non détectée
P2	Non-respect du contrat d'API ou de la qualité attendue, sans effet direct sur la décision
P3	Confort d'exploitation

Les tests P1 tournent à chaque Pull Request et empêchent la fusion s'ils échouent.

4. Matrice de traçabilité

ID	Exigence	Résultat attendu	Risque si défaillance	Priorité	Cas de test
EX01	L'âge est un entier compris entre 18 et 75 ans	Valeurs hors limites rejetées (422)	Scoring d'un mineur, donnée saisie erronée	P1	`test_ex01_*`, API03, API08, ROB04
EX02	Le revenu est numérique, fini et strictement positif	Erreur contrôlée 422 `float_type` ou `finite_number`	Division par zéro, conversion silencieuse d'un texte	P1	`test_ex02_*`, API04, ROB02, ROB03
EX03	La durée est un entier de 1 à 60 mois	12,5 et 12.0 rejetés (`int_type`)	Mensualité faussée	P1	`test_ex03_*`, API06, ROB04
EX04	La prédiction est une probabilité finie dans [0, 1]	`0 <= probabilite_defaut <= 1`	Décision incohérente	P1	API01, INT11, ROB07 (fuzzing), REG07
EX05	L'indisponibilité du modèle est signalée	503 `MODELE_INDISPONIBLE` sur les trois endpoints	Réponses fausses ou plantage silencieux	P1	API09, INT07 (absent, corrompu, empreinte, métadonnées, objet invalide)
EX06	Une entrée incomplète est rejetée	422 avec le champ manquant identifié	Imputation implicite non maîtrisée	P1	`test_ex06_*`, API02, ROB01, INT08
EX07	Une nouvelle version ne dégrade pas les critères critiques	Baisse de rappel et de F1 au plus 0,03, d'AUC au plus 0,02	Hausse des défauts non détectés	P1	REG01 à REG09, job `non-regression`
EX08	Les champs non prévus sont refusés	422 `extra_forbidden`	Introduction d'une variable sensible (sexe, ethnie) ou erreur d'intégration	P2	`test_ex08_*`, API07, ROB06
EX09	L'ancienneté est cohérente avec l'âge (au plus âge moins 14 ans)	422 si incohérente	Donnée frauduleuse ou erronée	P2	`test_ex09_*`, ROB04
EX10	Le montant est compris entre 10 000 et 500 000 000 FCFA	422 hors bornes	Montant absurde scoré (ANO-03)	P1	API05, API08, ROB04
EX11	Une entrée valide mais inhabituelle produit une alerte et une analyse manuelle	200, alerte `HORS_DOMAINE_ENTRAINEMENT`, décision `analyse_manuelle_hors_domaine`	Décision automatique sur une extrapolation (ANO-02, ANO-04)	P1	`test_ex11_*`, API08, ROB05

ID	Exigence	Résultat attendu	Risque si défaillance	Priorité	Cas de test
EX12	Toute erreur respecte le contrat `{erreur, id_requete}`	400 pour JSON illisible, 404, 405 au même format	Client incapable d'interpréter l'erreur (ANO-01)	P2	API10, API13, ROB03, ROB06
EX13	Les métadonnées décrivent l'artefact réellement chargé	Version, variables, seuil, métriques, SHA-256 cohérents	Mauvaise version en service sans le savoir	P2	API12, API15, INT06, REG02, REG05
EX14	Aucune trace technique n'est exposée ; chaque requête est traçable	Message générique, identifiant `X-Request-ID`	Fuite d'information exploitable	P1	`verifier_erreur_propre` (tous les tests d'erreur), INT09 à INT12, API14
EX15	Les résultats sont reproductibles	Même entrée, même sortie ; rechargement identique ; réentraînement identique	Résultats non auditables	P1	ROB07, REG04, REG06, `test_pretraitement_deterministe`, INT14
EX16	Le risque varie dans le sens métier attendu	Croît avec le montant, décroît avec le revenu et l'ancienneté (tolérance 0,02)	Refus injustifié de bons profils (ANO-02)	P2	ROB08
EX17	Performance adaptée à l'usage en agence	P95 inférieur à 200 ms à 20 requêtes simultanées, débit au moins 50 req/s, 0 erreur	Attente des agents, abandon de l'outil	P2	`scripts/benchmark.py`, `locustfile.py`
EX18	Chaque Pull Request est validée automatiquement	Ruff, Pytest, couverture au moins 90 %, échec bloquant	Régression fusionnée sans contrôle	P2	`.github/workflows/tests.yml`, `test_echec_volontaire`

5. Cas de test détaillés de l'API

Pour tous ces cas, le modèle 1.0.0 est chargé (sauf pour API09) et j'utilise le `TestClient` de FastAPI.

ID	Donnée d'entrée	Code attendu	Contenu attendu	Comportement de l'application
API01	Demandeur de référence (35 ans, 250 000, 1 000 000, 12, 4)	200	Sept champs dont `probabilite_defaut` dans [0, 1], `classe_risque`, `alertes` vide	Validation, prétraitement, inférence, classement
API02	Un des cinq champs absent	422	`DONNEES_INVALIDES`, détail `missing` sur le champ	Rejet avant prétraitement
API03	`age = -5`	422	Détail sur `age`	Rejet
API04	`revenu_mensuel = "250000"`, texte, chaîne vide	422	`float_type`, « Doit être un nombre »	Aucune conversion implicite (mode strict)
API05	`montant_credit` égal à 0, -1, -1 000 000	422	Détail sur `montant_credit`	Rejet
API06	`duree_mois` égal à 12.5, 12.0, "12"	422	`int_type`	Rejet
API07	Champ `sexe` ajouté	422	`extra_forbidden`	Rejet

ID	Donnée d'entrée	Code attendu	Contenu attendu	Comportement de l'application
API08	Valeurs de 10^4 à 10^15	422	`less_than_equal`	Rejet ; valeurs élevées mais valides acceptées avec alerte
API09	Modèle absent	503	`MODELE_INDISPONIBLE`	`/health`, `/predict`, `/model-info` signalent l'indisponibilité
API10	JSON tronqué, texte, liste, nombre, `null`, corps vide	400 ou 422	`REQUETE_MAL_FORMEE` ou `DONNEES_INVALIDES`	Rejet sans trace
API11	`GET /health`	200	`statut = ok`, version	
API12	`GET /model-info`	200	Version, algorithme, 8 variables, seuil, métriques, SHA-256	
API13	`GET /predict`, route inconnue	405, 404	Contrat d'erreur	
API14	En-tête `X-Request-ID`	200 ou 422	Identifiant renvoyé dans l'en-tête et le corps	Traçabilité dans les journaux
API15	`/health` et `/model-info`	200	Même version	